

Informe de Laboratorio 2

Branching, Pull Requests y Ejecución de CI



Módulo: Entornos de Integración y Entrega Continua (CI/CD)

Estudiante: René Velásquez

Correo: velasquez.rene@ficct.uagrm.edu.bo

Plataforma CI/CD: GitHub Actions

URL del Repositorio: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua>

URL del Pull Request #1: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/pull/1>

SANTA CRUZ - BOLIVIA

1. Introducción y Objetivos

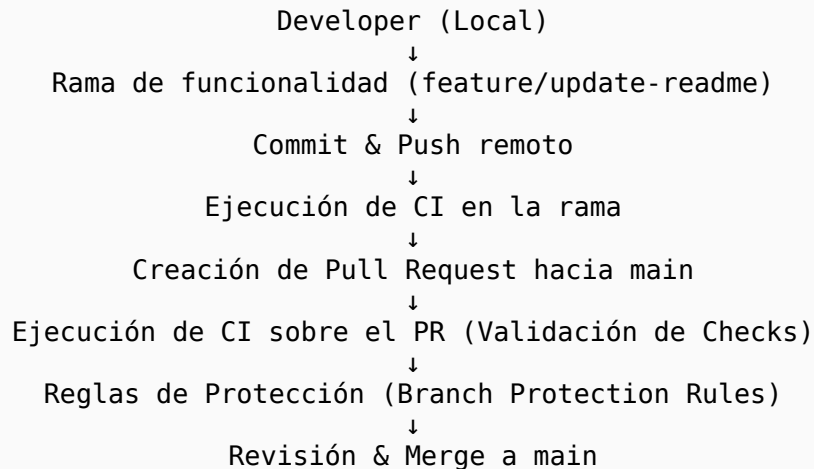
En este laboratorio se amplió el proyecto desarrollado en el Laboratorio 1, evolucionando desde un modelo simple basado únicamente en push directo hacia una estrategia de branching basada en ramas de funcionalidad (Feature Branching) integrada con Pull Requests (PR) y políticas de protección de la rama principal (main).

Objetivos Alcanzados

1. Creación y gestión de la rama de funcionalidad feature/update-readme.
- Implementación del flujo de integración controlada mediante Pull Request.
- Configuración de eventos múltiples en GitHub Actions (push y pull_request).
- Establecimiento de reglas de protección en la rama main (requiriendo PR y validación de status checks).
- Experimentación con fallos deliberados y recuperación automática de la salud del pipeline.

2. Flujo de Trabajo y Estrategia de Branching

El flujo implementado garantiza que ningún desarrollador introduzca cambios directamente sobre main, protegiendo la estabilidad del código en producción:



3. Pipeline as Code (.github/workflows/pipeline.yml)

El pipeline se actualizó para reaccionar a eventos en ramas de funcionalidad y Pull Requests:

```
name: Primer Pipeline CI

on:
  push:
    branches:
      - main
      - 'feature/**'
  pull_request:
    branches:
      - main

jobs:
  hello-ci:
    name: Hello CI
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Mostrar información del entorno
        run: |
          echo "=====
          echo "          PIPELINE CI"
          echo "=====
          echo "Repositorio: ${github.repository}"
          echo "Evento:      ${github.event_name}"
          echo "Rama/Ref:    ${github.ref_name}"
          echo "Commit SHA:  ${github.sha}"
          echo "=====

      - name: Mostrar fecha y hora
        run: date

      - name: Mostrar versión de Git
        run: git --version

      - name: Simulacion de verificacion y build
        run: |
          echo "Ejecutando verificacion de build y consistencia..."
          test -f README.md && test -f app/hello.txt
          echo "Validacion de archivos requeridos: EXITOSA (OK)"

      - name: Finalizar pipeline
        run: echo "Pipeline ejecutado correctamente."
```

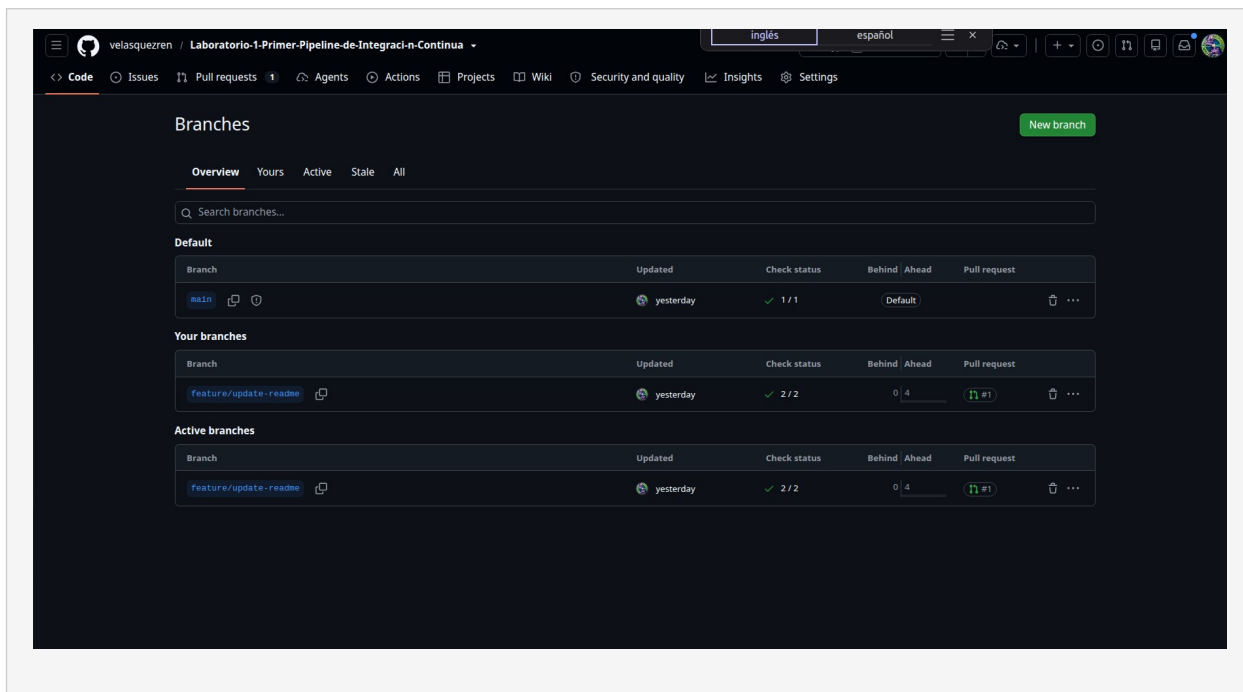
4. Evidencias de Ejecución (Capturas de Pantalla)

A continuación se detallan los enlaces y los recuadros donde insertar las capturas solicitadas en los entregables:

Captura 1: Creación y Existencia de la Rama de Funcionalidad

Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/branches>

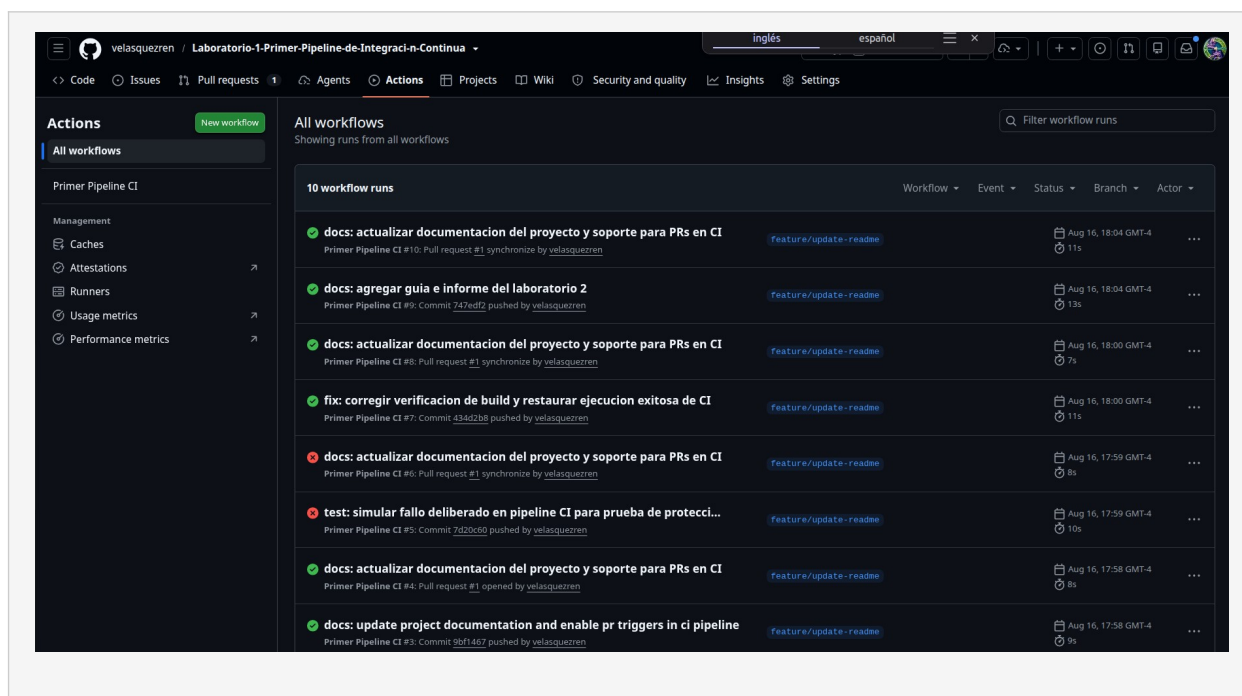
Descripción: muestra la lista de ramas activas en GitHub, evidenciando la rama feature/update-readme creada a partir de main.



Captura 2: Ejecución del Pipeline en la Rama y en el Pull Request

Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/actions>

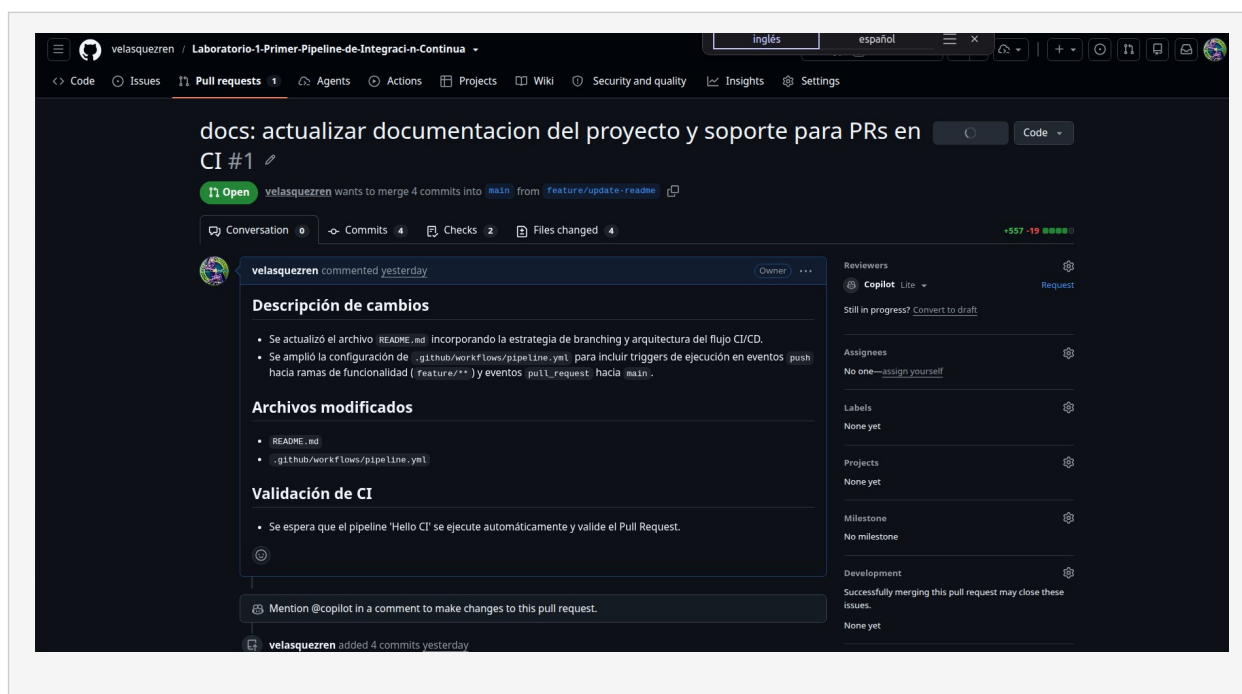
Descripción: muestra las ejecuciones del workflow en la pestaña Actions, evidenciando los eventos push y pull_request sobre la rama de funcionalidad.



Captura 3: Creación y Estado del Pull Request

Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/pull/1>

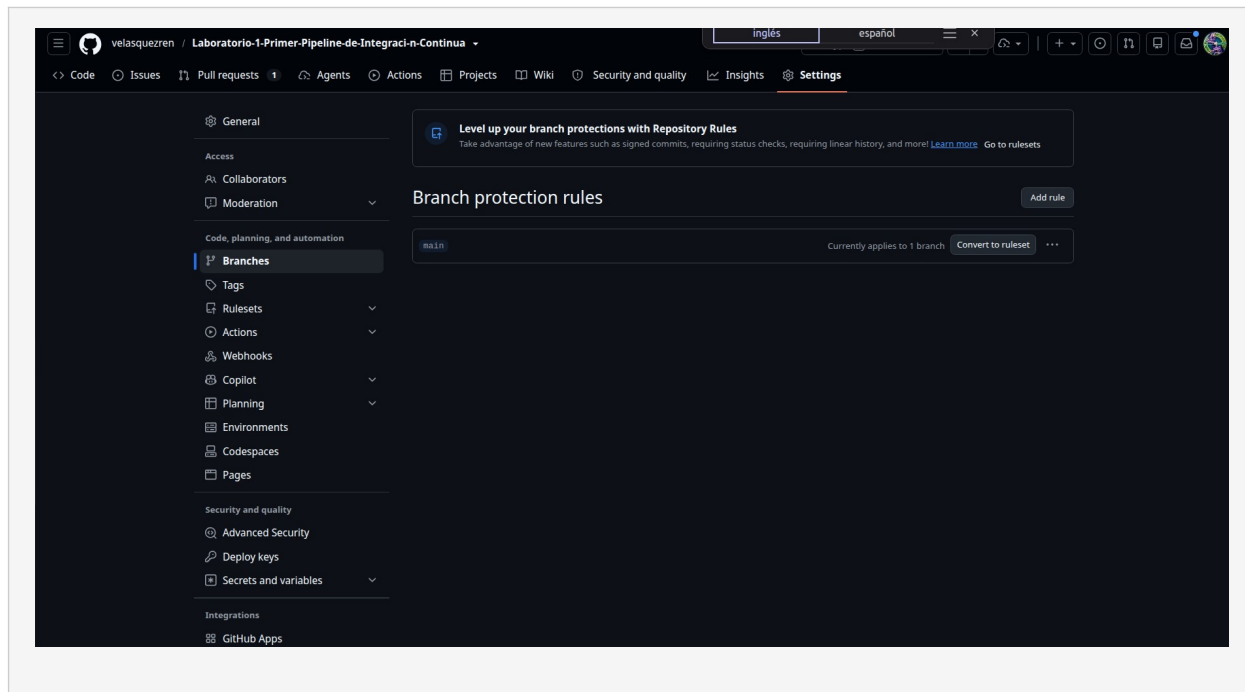
Descripción: muestra el Pull Request #1 abierto desde feature/update-readme hacia main con su descripción y lista de commits.



Captura 4: Reglas de Protección de la Rama Principal (main)

Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/settings/branches>

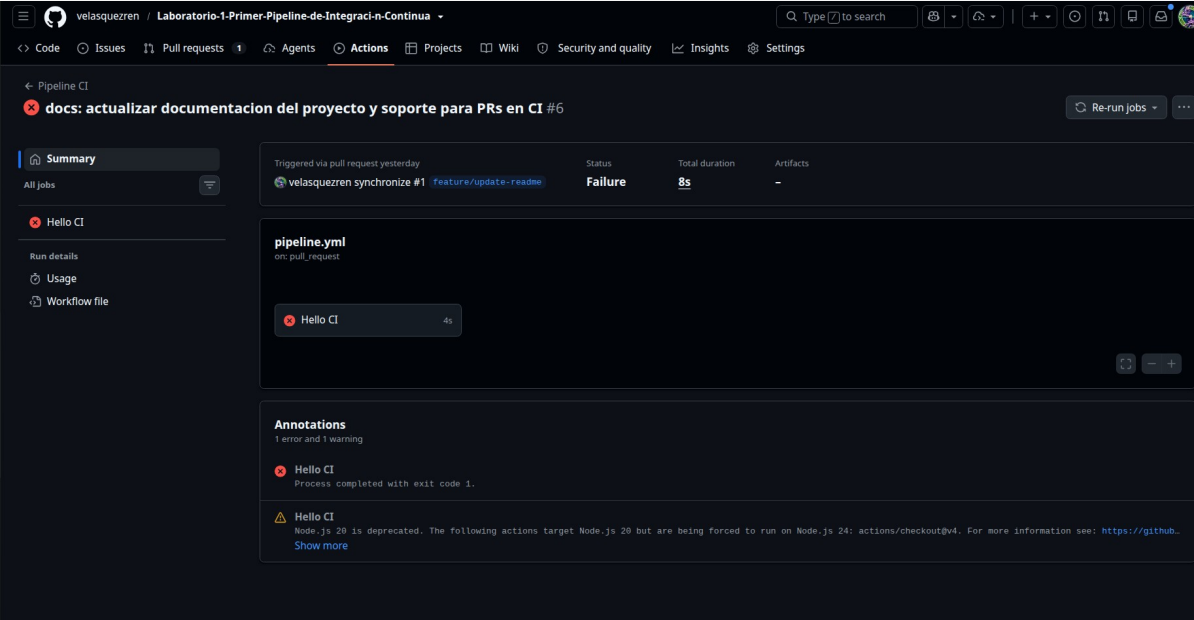
Descripción: muestra la configuración de Branch Protection Rules para la rama main, exigiendo Pull Request y el check obligatorio Hello CI.



Captura 5: Simulación de Ejecución Fallida (Fallo Deliberado)

Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/actions/runs/31975074208>

Descripción: muestra el fallo simulado en el pipeline (marca roja) y el bloqueo del Pull Request al no cumplirse el status check requerido.

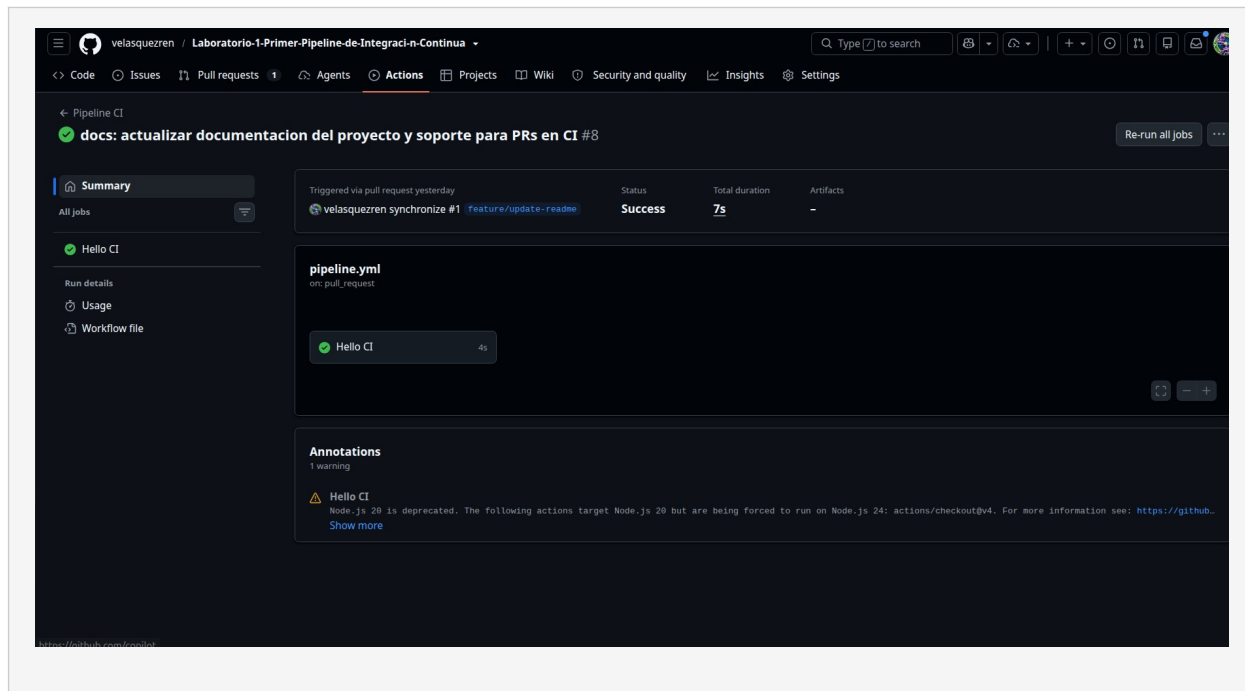


The screenshot displays the GitHub Actions interface for a workflow named "Pipeline CI". The workflow is triggered by a pull request and is currently in a "Failure" state. The status bar at the top indicates "docs: actualizar documentacion del proyecto y soporte para PRs en CI #6" with a red "X" icon. The left sidebar shows the "Summary" tab selected, with options for "All jobs", "Hello CI", "Run details", "Usage", and "Workflow file". The main content area shows the workflow details, including the trigger "Triggered via pull request yesterday" and the status "Failure" with a total duration of "8s". Below this, the "pipeline.yml" file is shown, indicating it was triggered on "pull_request". A job named "Hello CI" is shown with a duration of "4s" and a red "X" icon, indicating failure. The "Annotations" section shows "1 error and 1 warning". The error message states: "Hello CI: Process completed with exit code 1." The warning message states: "Hello CI: Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: https://github.com/velasquezren/synchronize#1. Show more".

Captura 6: Ejecución Corregida y Estado Exitoso

Enlace directo: <https://github.com/velasquezren/Laboratorio-1-Primer-Pipeline-de-Integraci-n-Continua/actions/runs/31975118120>

Descripción: muestra la recuperación del pipeline tras el commit de corrección (check verde) y el Pull Request listo para ser integrado.



5. Respuestas a las Preguntas de Análisis

Pregunta Inicial (Parte 1)

¿Qué evento provoca actualmente la ejecución del pipeline?

Respuesta: en el Laboratorio 1, el pipeline se disparaba exclusivamente ante eventos de tipo push directo sobre la rama main. Tras la evolución en este Laboratorio 2, el pipeline se dispara ante eventos de push (tanto en main como en ramas feature/**) y ante eventos de pull_request dirigidos a main.

Preguntas de Flujo (Parte 6)

1. ¿Por qué es conveniente trabajar en una rama independiente?

Trabajar en ramas de funcionalidad (feature branches) aísla el código en desarrollo respecto a la versión estable de producción (main). Esto previene que código incompleto o inestable afecte a otros miembros del equipo, facilita el desarrollo paralelo de múltiples funcionalidades y permite realizar experimentos o pruebas de forma segura sin comprometer el despliegue principal.

2. ¿Qué ventaja proporciona realizar el Pull/Merge Request antes del merge?

El Pull Request actúa como una puerta de control de calidad (Quality Gate) y colaboración. Permite:

- Ejecutar pruebas y validaciones automáticas de CI sobre la combinación del código propuesto antes de incorporarlo.
- Facilitar la revisión por pares (Code Review), discusiones y sugerencias de mejora.
- Mantener una trazabilidad histórica del por qué y cómo se introdujo cada cambio.
- Aplicar políticas organizacionales obligatorias (aprobaciones, firmas, pruebas pasando).

3. ¿En qué momento se ejecutó el pipeline?

El pipeline se ejecutó en dos momentos clave:

1. Al hacer push a la rama remota feature/update-readme: validando los cambios de forma temprana en el entorno del desarrollador.
2. Al abrir/actualizar el Pull Request: validando la integración potencial contra la rama main.

4. ¿Qué ocurriría si el pipeline fallara?

Si el pipeline falla (como se demostró en la Parte 8):

- La plataforma marca el status check como FAILURE / FAILED.
- Gracias a las reglas de protección de rama, el botón de Merge queda bloqueado, impidiendo que cualquier desarrollador fusione código defectuoso a main.
- Se generan logs detallados indicando la causa raíz del fallo para que el autor suba un nuevo commit correctivo sobre la misma rama.

5. ¿Qué diferencia existe entre revisar código manualmente y validarlo mediante CI?

Criterio	Revisión Manual (Code Review)	Validación Automatizada (CI)
Enfoque	Diseño, arquitectura, legibilidad, lógica de negocio y buenas prácticas.	Sintaxis, compilación, ejecución de tests, análisis estático, seguridad y formateo.
Velocidad	Lenta (depende de la disponibilidad del revisor humano).	Rápida e inmediata (minutos/segundos tras el push).
Consistencia	Subjetiva y propensa a descuidos u omisiones humanas.	100% determinística y estandarizada en cada ejecución.

Ambos enfoques se complementan: el CI valida que el código funcione técnicamente para que el revisor humano se concentre en el valor del código.

6. Conclusiones

1. La combinación de Feature Branching + Pull Requests + CI/CD constituye el estándar de la industria para el desarrollo colaborativo y seguro de software.

Las Branch Protection Rules aseguran que las directrices de calidad no dependan de la disciplina manual, sino que sean impuestas automáticamente por la plataforma.

El pipeline asume el rol de centinela del repositorio, permitiendo una detección temprana de errores (Shift-Left Testing) antes de que impacten ramas productivas.